



KOLEJ PROFESIONAL MARA INDERA MAHKOTA
DIPLOMA IN COMPUTER SCIENCE

COURSE NAME	:	OBJECT ORIENTED PROGRAMMING
COURSE CODE	:	CSC2923
ACADEMIC SESSION	:	SESSION 3 2025/2026
TYPE OF ASSESSMENT	:	FINAL ASSIGNMENT
DURATION	:	4/2/2026-23/2/2026

CLO 3: Employ third party data in object oriented application development using graphical user interface (GUI) application framework

INSTRUCTION TO CANDIDATES:

1. Late submissions after the given due date will not be accepted.
2. Report should be written using: Font type: Arial

Size: 12 pts

Line Spacing: 1.5

3. Coding format:

Font type: Consolas

Size: 10 pts

Line Spacing: Single

Personal Details	
Name	FARAH NABIHA BINTI AZHAR
I/D Number	ICS24-03-027
Class	DCS 4A
Lecturer	MADAM ZULIARTY

Section / Question No.	Marks
1	
2	
3	
4	
5	
6	
Total	/ 50

Weather analytics and forecast decision support

TABLE OF CONTENT

Content	Pages
Overview of application	1
System functionality	2-3
Interface documentation	4-8
Code highlight	9-11
Conclusion	12
Appendices	13-31

OVERVIEW OF APPLICATION

The name of the application that has been developed is WeatherCast. The problem that users usually face is that they find it difficult to make decisions based solely on raw weather data. However, with WeatherCast, it makes it easier for users to make decisions and provides suggestions, such as recommending activities that users can do based on the current weather conditions. In addition, users can manage their own activities according to their preferences. Furthermore, WeatherCast makes it easy for users to see the average temperature clearly and easily. The target user for this application are everyone who wants to search for weather and get helps in decision making.

This application integrates the Weather API to retrieve forecast weather data based on user input. The system fetches important data such as activities details, average temperature, risk of rain, suggestion outdoor or indoor and weather condition.

The purpose of the application is transform basic weather analysis by functioning as a comprehensive Decision Support System (DSS). This application is to facilitate safer and more efficient daily planning. By integrating predictive analytics, this application helps users to mitigate risks when planning outdoor activities. Next is this application helps users avoid dangerous weather such as heavy rain or storms during outdoor activities and it suggests what activities are suitable for the current weather, such as indoor or outdoor activities.

Summary for the objective of application :

1. To helps uers in making decisions by analysing weather data and provide activity recommendations
2. To allow users to manage their personalized activities through CRUD functionalities
3. To helps users plan their daily activities by calculate the average of temperatures, analysing isk of rain and compare the weather between two locations
4. To provide users a user friendly interfaces that displays clear informations.

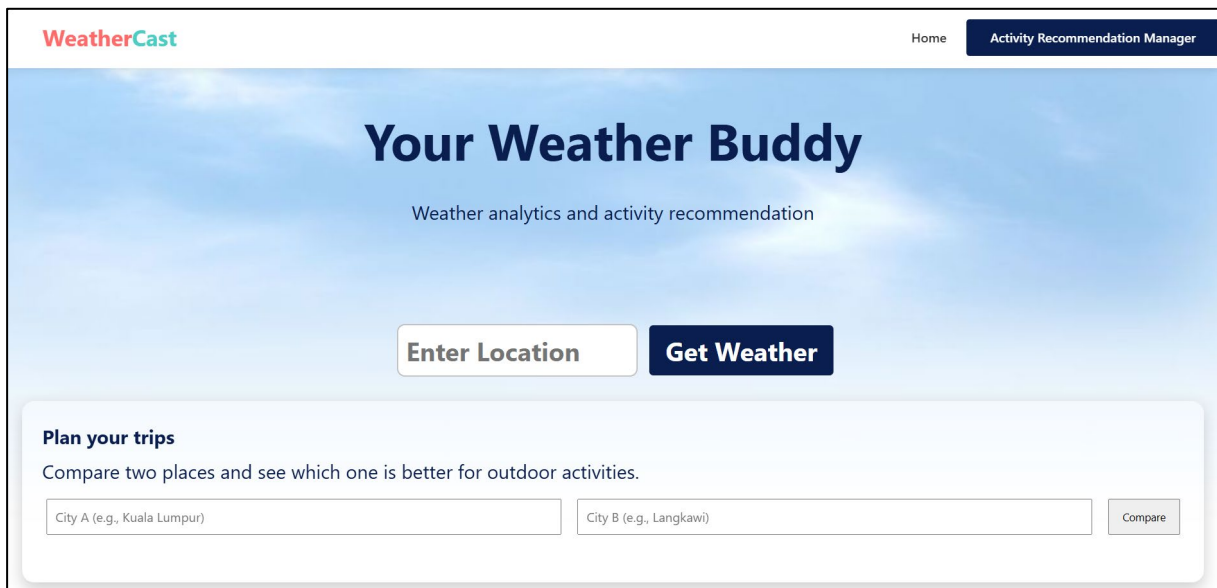
SYSTEM FUNCTIONALITY

Features	Description
Search for location	Users can search for a location that they wanted to know the current weather condition and forecast analysis. The application connects to the API weather and retrieves the data such as current temperature and humidity
City comparison	Users can compare two different location to determine which location is more suitable for their trip plans. The system retrieves weather data for both cities and determine the best option based on the analysis
Average temperature Calculation	The application calculates the average temperature based on forecast data retrieved from the API. This helps users understand overall weather trends.
Risk of rain analysis	The weather data get from the API will determine the potential risk of rain. This helps users decide whether outdoor activities are suitable or not
Indoor or outdoor activity recommendation	Based on temperature and weather condition, the system will recommends suitable indoor or outdoor activities.
Activity manager (CRUD)	Users can add new activities, view saved activities, update activities details, and delete the activities. This allows users to personalize and manage their activity.
Users Feedback	The system provides feedback message such as "Activity added successfully" to inform users that their action have been completed.
Data stored	Data such as average temperature, risk of rain, suggestion indoor or outdoor, weather condition and all activities are stored locally using JSON format in local storage. This will ensures that the data remain available even after the application is closed
Navigation and Usability	The application contains two pages for weather analysis and activity management. There is also button for the

	navigation that helps users move easily between pages.
User-Friendly interface design	The application uses structured layout, readable fonts, and appropriate color to enhance usability and improves users experience

INTERFACE DOCUMENTATION

1. The first pages is to show the weather and forecast analysis, and weather comparison between two locations.



WeatherCast Home Activity Recommendation Manager

Your Weather Buddy

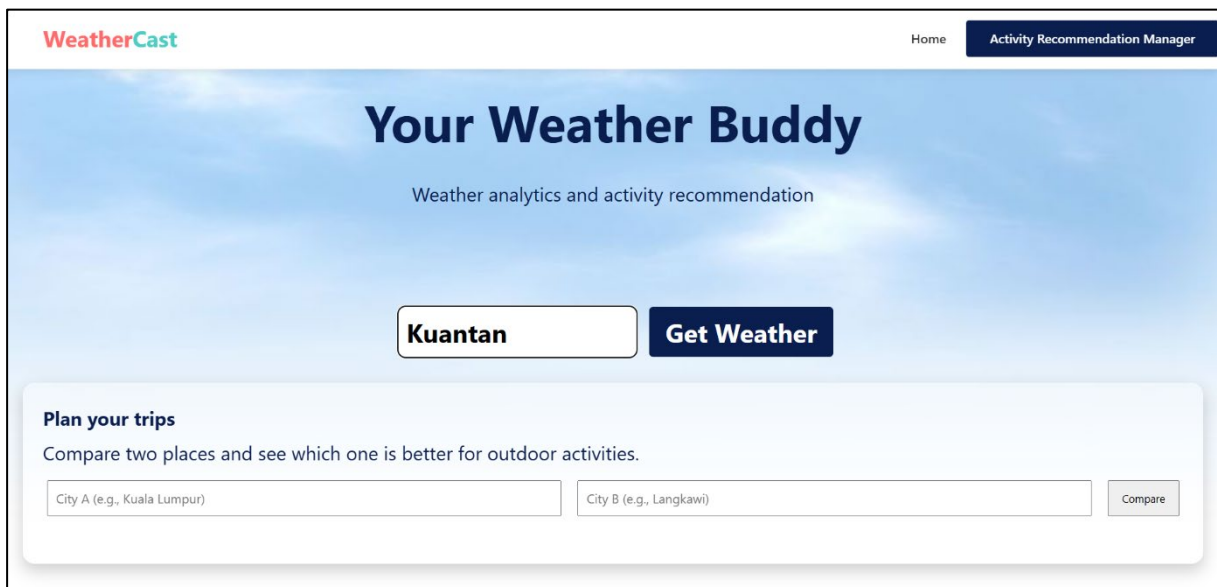
Weather analytics and activity recommendation

Enter Location Get Weather

Plan your trips
Compare two places and see which one is better for outdoor activities.

City A (e.g., Kuala Lumpur) City B (e.g., Langkawi) Compare

Figure 1 : Main page



WeatherCast Home Activity Recommendation Manager

Your Weather Buddy

Weather analytics and activity recommendation

Kuantan Get Weather

Plan your trips
Compare two places and see which one is better for outdoor activities.

City A (e.g., Kuala Lumpur) City B (e.g., Langkawi) Compare

Figure 2 : Location input

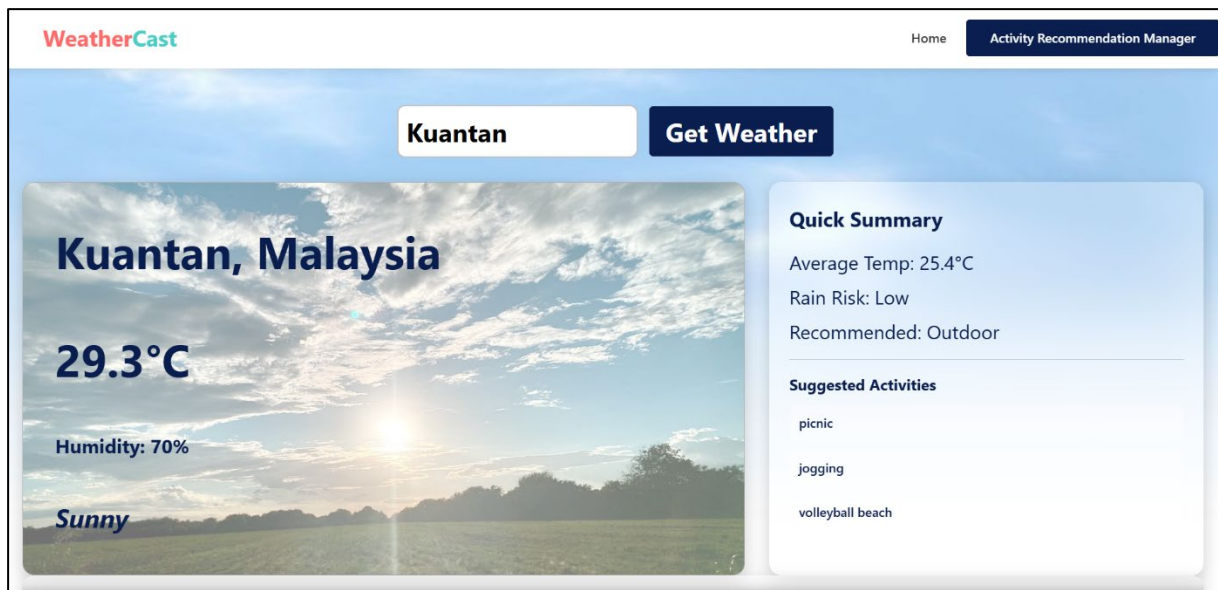


Figure 3 : The output displayed based on users location input

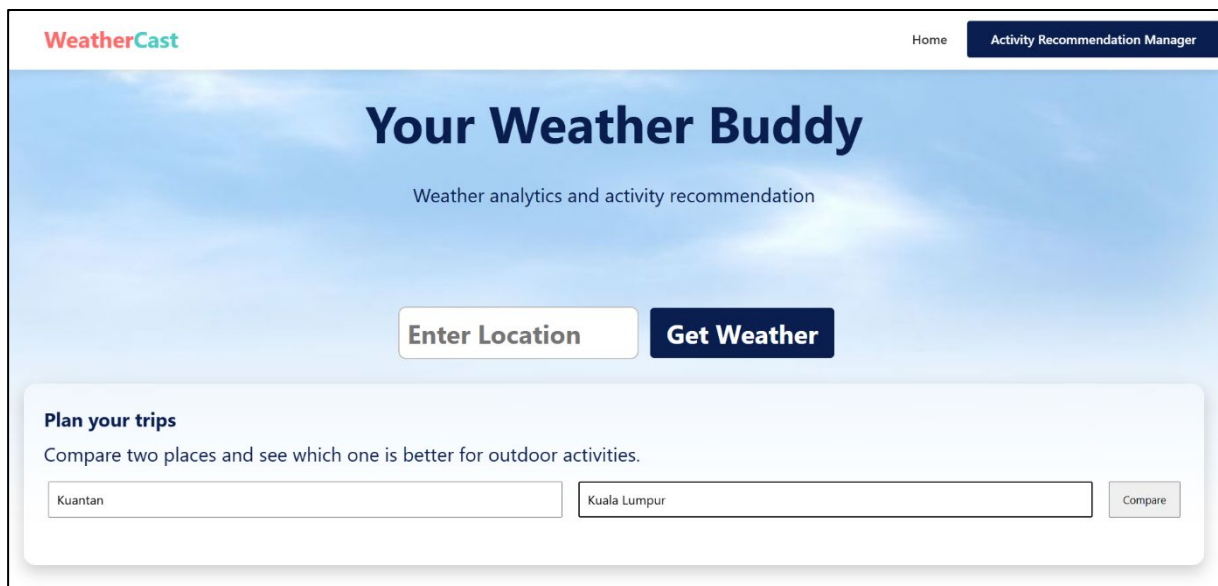


Figure 4 : Comparison between location forms

WeatherCast

Home Activity Recommendation Manager

Weather analytics and activity recommendation

Enter Location Get Weather

Plan your trips
Compare two places and see which one is better for outdoor activities.

Kuantan Kuala Lumpur Compare

Kuantan: Suitable. Temp 25.3°C, Rain 0%

Kuala Lumpur: Not suitable - high chance of rain. Temp 25.7°C, Rain 90%

Better for outdoor today: ✓ Kuantan

Figure 5 : Output based on users input

From the figure 1, users will see this interfaces when they first open the application, Next is in figure 2, users will enter their location, and from figure 3 it will displays the output for weather and forecast analysis based on users input. In figure 4, users can compare the location that they wanted to go on trips, so that they can know the current weather and avoid from bad weather. Lastly, in figure 5, after users enter their location, the output will be displayed.

2. The second pages helps users manage their preferences activity

WeatherCast

Home Activity Recommendation Manager

Activity Recommendation

Activity Name Indoor Add

No activities yet. Add one above.

Figure 6 : Second page

WeatherCast

HomeActivity Recommendation Manager

Activity Recommendation

Activity added successfully

uno card (Indoor)	Edit	Delete
picnic (Outdoor)	Edit	Delete
volleyball (Outdoor)	Edit	Delete

Figure 7 : Added activities

WeatherCast

HomeActivity Recommendation Manager

Activity Recommendation

Activity updated

uno card games (Indoor)	Edit	Delete
picnic (Outdoor)	Edit	Delete
volleyball (Outdoor)	Edit	Delete

Figure 8 : Edit activity

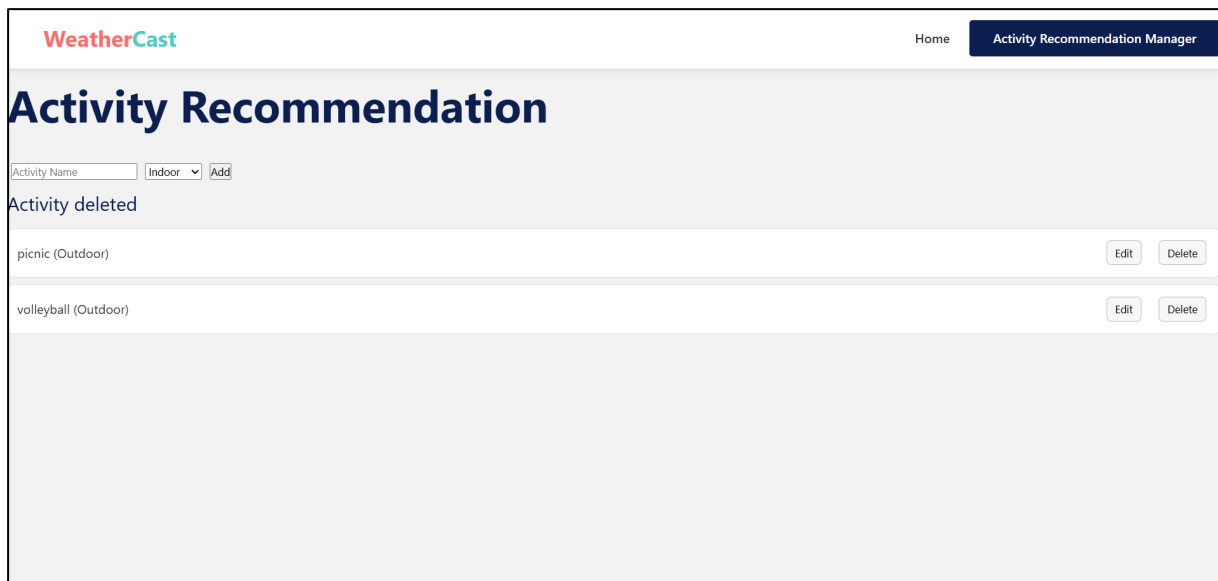


Figure 9 : Delete activity

From the figure 6, users will view this interface when users click Activity Recommendation Manager button, which is the button navigate from the first page to the second page. Next in Figure 7, users can added their preferred activities in the search bar and the click the add button to add the activity into the list. After that, in figure 8, users also can edit their activity, for example the activity is misspelled, so users can correct it using the edit button. Lastly, in the figure 9, users can delete the unwanted activities from the list.

CODE HIGHLIGHTS

```
localStorage.setItem("weatherCondition", data.current.condition.text);
localStorage.setItem("avgTemp", avgTemp);
localStorage.setItem("rainRisk", risk);
localStorage.setItem("suggestionType", suggestion); // "Indoor" / "Outdoor"
```

Figure 10 : save data into local storage

This code is to stored weather data and user activities locally using local storage in JSON format.

```
function saveActivities() {
  localStorage.setItem("activities", JSON.stringify(activities));
}
```

Figure 11 : Save activities function

The saveActivities function converts the activities array into JSON string and saves it in local storage.

```
// fetch data for get weather form
async function getweatherData(city) {

  const apiUrl = `https://api.weatherapi.com/v1/forecast.json?key=${apiKey}&q=${city}&days=1`;

  const response = await fetch(apiUrl);

  if (!response.ok) { // error if city not found
    throw new Error("City not found");
  }

  const data = await response.json();
  console.log(data);
  return data;
}
```

Figure 12 : Fetch data

This function is to constructs the weather API request URL based on the user's input location. It will send fetch() request and check the reponses. If the city is invalid, it will display error message. And if successful, the data will returned for the next process.

```
// calculate Average temp
let total = 0;
hours.forEach(h => total += h.temp_c);
const avgTemp = (total / hours.length).toFixed(1);
```

Figure 13 : Calculate average temperature

This code is to calculate average temperature using hourly forecast data which is 24 hours. The results is in one decimal place using toFixed(1)

```
// Rain risk
const rainy = hours.filter(h => h.chance_of_rain >= 60).length;

// determine rain risk level
let risk = "Low";
if(rainy >= 8) risk = "High";
else if(rainy >= 3) risk = "Medium";
```

Figure 14 : Analysis risk of rain

The risk of rain is categorized into low, medium or high. The systems will count how many hourly forecasts have chance_of_rain greater or equal to 50 percent.

```
// add activity process
const newActivity = { // create new object
  id: Date.now(),
  name,
  type
};

activities.push(newActivity); // added into the list
saveActivities();
renderActivities();
resetForm();
showFeedback("Activity added successfully");
});
```

Figure 15 : CRUD process (add)

A new activity is create using Date.now() as ID, then it will added to the activities array. The updated lis is saved to local storage

```
// delete process
const delBtn = document.createElement("button");
delBtn.textContent = "Delete";
delBtn.onclick = () => {
  activities = activities.filter(a => a.id !== activity.id);
  saveActivities();
  renderActivities();
}
```

Figure 16 : CRUD process (delete)

The selected activity is removed using the filter() method based on its ID. The updated lists is saved to local storage.

```
// to display message in the interface
const feedbackMsg = document.getElementById("feedbackMsg");
function showFeedback(msg){
  feedbackMsg.textContent = msg;

  setTimeout(function(){
    feedbackMsg.textContent = "";
  }, 2000); // set time, after 2 sec, it will disappear
}
```

Figure 17 : Users feedback message

The function displays a temporary message on the interface such as “Activity added successfully” to inform users that their action was completed. The messages will disappear after two seconds.

CONCLUSION

The WeatherCast application successfully demonstrates the integration of Object-Oriented Programming principles with modern desktop development frameworks such as Electron.js and Node.js. The journey through the integration of Weather API, the system is able to forecast data and apply logic such as calculation of average temperature, risk of rain analysis and location comparison to helps users in making decision.

The application also implements CRUD functions, which allows users to manage their own preferred activity with persistent local storage. This enables personalization and the users feedback features enhance usability by providing clear confirmation of actions for users. Furthermore, the systems includes features such as location comparison and activity recommendation based on the current weather. These demonstrate the ability to performs raw API data into meaningful insight

Overall, the project achieves the objective learning with applying third-party API integration within a GUI-based electron application while incorporating data manipulation, user interaction and local storage.

SYSTEM CODE**Main.js**

```
const { app, BrowserWindow } = require('electron');
const fs = require('fs')
const path = require('path');

let mainWindow;

function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1200,
    height: 800,
    webPreferences: {
      nodeIntegration: true,
      contextIsolation: false,
    }
  });

  mainWindow.maximize()
  mainWindow.loadFile('src/index.html'); //first page
}

app.whenReady().then(createWindow);

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') app.quit();
});

app.on('activate', () => {
  if (BrowserWindow.getAllWindows().length === 0) createWindow();
});
```


Index.js

```

const weatherForm = document.querySelector(".weatherForm");
const cityInput = document.querySelector(".cityInput");
const card = document.querySelector(".card");
const apiKey = "32804b24a847407391c53709241010";

// Compare location
const cityA = document.querySelector(".cityA");
const cityB = document.querySelector(".cityB");
const compareBtn = document.querySelector(".compareBtn");
const compareResult = document.querySelector(".compareResult");

weatherForm.addEventListener("submit", async event => {

    event.preventDefault();

    const city = cityInput.value;
    // if city is entered, fetch weather data
    if(city){
        try{
            const weatherData = await getweatherData(city);
            displayWeatherInfo(weatherData);
        }
        catch(error){
            console.error(error);
            displayError(error.message);
        }
    }
    else{
        displayError("Please enter location")
    }
});

// function compare button click (cityA vs cityB)
compareBtn.addEventListener("click", async () => {
    const a = cityA.value.trim();
    const b = cityB.value.trim(); // trim is to removes extra spaces

    // ensure both has input
    if (!a || !b) {
        compareResult.innerHTML = `

Please enter both City A and City B.</p>`;
        return;
    }

    try { // fetch weather for both cities
        const dataA = await getData(a);
        const dataB = await getData(b);

        // check outdoor suitability
        const checkA = outdoorCheck(dataA);
        const checkB = outdoorCheck(dataB);

        let win = "";

        // decide which city is better
        if (checkA.ok && !checkB.ok) win = dataA.location.name;
        else if (!checkA.ok && checkB.ok) win = dataB.location.name;
    }
});


```

```

    else win = checkA.rain < checkB.rain ? dataA.location.name :
dataB.location.name;

    // display result
    compareResult.innerHTML = `
    <p><b>${dataA.location.name}</b>: ${checkA.msg}</p>
    <p><b>${dataB.location.name}</b>: ${checkB.msg}</p>
    <hr>
    <p><b>Better for outdoor today:</b> ${win}</p>
    `;
  } catch (err) {
    compareResult.innerHTML = `<p>Error: ${err.message}</p>`;
  }
});

// fetch data for get weather form
async function getweatherData(city) {

  const apiUrl =
`https://api.weatherapi.com/v1/forecast.json?key=${apiKey}&q=${city}&days=1`;

  const response = await fetch(apiUrl);

  if (!response.ok) { // error if city not found
    throw new Error("City not found");
  }

  const data = await response.json();
  console.log(data);
  return data;
}

function displayWeatherInfo(data){ // display data

  card.textContent = "";
  card.style.display = "flex";

  const cityDisplay = document.createElement("h1");
  const tempDisplay = document.createElement("p");
  const humidityDisplay = document.createElement("p");
  const descDisplay = document.createElement("p");

  // set weather values
  cityDisplay.textContent = `${data.location.name}, ${data.location.country}`;
  tempDisplay.textContent = `${data.current.temp_c}°C`;
  humidityDisplay.textContent = `Humidity: ${data.current.humidity}%`;
  descDisplay.textContent = data.current.condition.text;

  // change background image based on weather condition
  const condition = data.current.condition.text.toLowerCase();

  if(condition.includes("rain")){
    card.style.backgroundImage = "url('./Images/rain.jpg')";
  }
  else if(condition.includes("cloud")){
    card.style.backgroundImage = "url('./Images/cloudy.jpg')";
  }
  else if(condition.includes("fog") || condition.includes("mist")){
    card.style.backgroundImage = "url('./Images/foggysky.jpg')";
  }
}

```

```

    }
    else if(condition.includes("snow")){
        card.style.backgroundImage = "url('./Images/snow.jpg')";
    }
    else if(condition.includes("storm") || condition.includes("thunder")){
        card.style.backgroundImage = "url('./Images/storm.jpg')";
    }
    else{
        card.style.backgroundImage = "url('./Images/sunny.jpg')";
    }

    // styling
    card.style.backgroundSize = "cover";
    card.style.backgroundPosition = "center";
    card.style.backgroundRepeat = "no-repeat";

    cityDisplay.classList.add("cityDisplay");
    tempDisplay.classList.add("tempDisplay");
    humidityDisplay.classList.add("humidityDisplay");
    descDisplay.classList.add("descDisplay");

    card.appendChild(cityDisplay);
    card.appendChild(tempDisplay);
    card.appendChild(humidityDisplay);
    card.appendChild(descDisplay);

    if(!data.forecast) return;
    const hours = data.forecast.forecastday[0].hour;

    // calculate Average temp
    let total = 0;
    hours.forEach(h => total += h.temp_c);
    const avgTemp = (total / hours.length).toFixed(1);

    // Rain risk
    const rainy = hours.filter(h => h.chance_of_rain >= 60).length;

    // determine rain risk level
    let risk = "Low";
    if(rainy >= 8) risk = "High";
    else if(rainy >= 3) risk = "Medium";

    // suggest activity
    let suggestion = "Outdoor";

    if(risk === "High" || avgTemp >= 32){
        suggestion = "Indoor";
    }

    // display summary
    document.getElementById("avgTemp").textContent =
    `Average Temp: ${avgTemp}°C`;

    document.getElementById("rainRisk").textContent =
    `Rain Risk: ${risk}`;

    document.getElementById("activitySuggest").textContent =
    `Recommended: ${suggestion}`;

```

```

    document.getElementById("summaryBox").style.display = "block";

    localStorage.setItem("weatherCondition", data.current.condition.text);
    localStorage.setItem("avgTemp", avgTemp);
    localStorage.setItem("rainRisk", risk);
    localStorage.setItem("suggestionType", suggestion); // "Indoor" / "Outdoor"

    renderRecommendedActivities(suggestion);
}

function renderRecommendedActivities(suggestionType){
    const listEl = document.getElementById("recommendedList");
    if(!listEl) return;

    const activities = JSON.parse(localStorage.getItem("activities")) || [];

    const filtered = activities.filter(a => a.type === suggestionType);

    listEl.innerHTML = "";

    if(filtered.length === 0){
        listEl.innerHTML = `<li>No saved ${suggestionType} activities yet. Add some
in Activity Manager.</li>`;
        return;
    }

    // pick 3 suggested activity only
    // use math random function to suggested activities randomly
    const shuffled = filtered.sort(() => 0.5 - Math.random());
    shuffled.slice(0,3).forEach(a => {
        const li = document.createElement("li");
        li.textContent = a.name;
        listEl.appendChild(li);
    });
}

// to display error message
function displayError(message){
    const errorDisplay = document.createElement("p");
    errorDisplay.textContent = message;
    errorDisplay.classList.add("errorDisplay");

    card.textContent = ""; // clear content inside box
    card.style.display = "flex";
    card.appendChild(errorDisplay); // append error message into card

    document.getElementById("avgTemp").textContent = "";
    document.getElementById("rainRisk").textContent = "";
    document.getElementById("activitySuggest").textContent = "";

    const listEl = document.getElementById("recommendedList");
    if(listEl) listEl.innerHTML = "";
    document.getElementById("summaryBox").style.display = "none";

    card.style.backgroundImage = "none";
    card.style.backgroundColor = "";
    card.style.backgroundBlendMode = "";

    localStorage.removeItem("weatherCondition");

```

```

}

// fetch weather for comparison features
async function getData(city) {
  const url =
`https://api.weatherapi.com/v1/forecast.json?key=${apiKey}&q=${encodeURIComponent(
t(city))&days=1`;
  const res = await fetch(url);
  if (!res.ok) throw new Error(`City not found: ${city}`);
  return await res.json();
}

// check outdoor suitability
function outdoorCheck(data) {
  const day = data.forecast.forecastday[0].day;

  const temp = day.avgtemp_c;
  const rain = day.daily_chance_of_rain ?? 0;

  let ok = true;
  let reason = "Suitable";

  if (rain >= 60) {
    ok = false;
    reason = "Not suitable - high chance of rain";
  } else if (temp > 33) {
    ok = false;
    reason = "Not suitable - hot weather";
  }

  // display temp + rain data
  const msg = `${reason}. Temp ${temp}°C, Rain ${rain}%`;

  return { ok, rain, temp, msg };
}

```

Page2.js

```

let activities = JSON.parse(localStorage.getItem("activities")) || [];

const suggestionType = localStorage.getItem("suggestionType"); // "Indoor" /
"Outdoor"

// get elements for CRUD process
const addButton = document.getElementById("addButton");
const nameInput = document.getElementById("activityName");
const typeInput = document.getElementById("activityType");
const activityList = document.getElementById("activityList");

// edit process
let editMode = false;
let activityEditingId = null; // to save activities that has been edited

function saveActivities() {
  localStorage.setItem("activities", JSON.stringify(activities));
}

function renderActivities() { // function to display activity lists
  activityList.innerHTML = "";
  // message will appear when no input
  if (activities.length === 0) {
    activityList.innerHTML = `<p>No activities yet. Add one above.</p>`;
    return;
  }

  activities.forEach(activity => {
    const row = document.createElement("div"); // create a container for an
    activity

    row.className = "activityItem";

    row.textContent = `${activity.name} (${activity.type})`;

    // edit process
    const editBtn = document.createElement("button");
    editBtn.textContent = "Edit";
    // when user click edit, the selected acitivity will loaded into form
    editBtn.onclick = () => {
      nameInput.value = activity.name;
      typeInput.value = activity.type;

      editMode = true;
      activityEditingId = activity.id;

      addButton.textContent = "Save"; // click save to update
      nameInput.focus();
    };

    // delete process
    const delBtn = document.createElement("button");
    delBtn.textContent = "Delete";
    delBtn.onclick = () => {
      activities = activities.filter(a => a.id !== activity.id);
      saveActivities();
      renderActivities();
    };
  });
}

```

```

        // kalau tengah edit item yang sama, reset back
        if (editMode && activityEditingId === activity.id) {
            resetForm();
        }
    };

    // to put edit and delete button into row
    row.appendChild(editBtn);
    row.appendChild(delBtn);
    // appendchild row to put it into activitylist to display to user
    activityList.appendChild(row);
    showFeedback("Activity deleted");
});
}

// to clear input
function resetForm() {
    editMode = false;
    activityEditingId = null;
    addButton.textContent = "Add";
    nameInput.value = "";
    typeInput.value = "Indoor";
}

// when click add and save button :
addButton.addEventListener("click", () => {
    // get user input
    const name = nameInput.value.trim();
    const type = typeInput.value;

    if (name === "") return;

    // update mode
    if (editMode) { // user want to save back the activity that has been edited
        const target = activities.find(a => a.id === activityEditingId);
        if (!target) {
            resetForm();
            return;
        }

        // update old data to new
        target.name = name;
        target.type = type;

        // save
        saveActivities();
        renderActivities();
        resetForm();
        showFeedback("Activity updated");
        return;
    }

    // add activity process
    const newActivity = { // create new object
        id: Date.now(),
        name,
        type
    };

```

```
activities.push(newActivity); // added into the list
saveActivities();
renderActivities();
resetForm();
showFeedback("Activity added successfully");
});

// to display message in the interface
const feedbackMsg = document.getElementById("feedbackMsg");
function showFeedback(msg){
    feedbackMsg.textContent = msg;

    setTimeout(function(){
        feedbackMsg.textContent = "";
    }, 2000); // set time, after 2 sec, it will disappear
}

// always display activities that existed in localStorage
renderActivities();
```


Index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>WeatherCast Application </title>
  <link rel="stylesheet" href="./css/style.css">
  <link rel="stylesheet" href="./css/dashboard.css">
</head>
<body>
  <header>
    <div class="container-header">
      <div class="logo">Weather<span>Cast</span></div>
      <nav>
        <a href="index.html">Home</a>
        <a href="Pages/page2.html" class="btn">Activity Recommendation
Manager</a>
      </nav>
    </div>
  </header>

  <style>
    html, body {
      margin: 0;
      padding: 0;
      height: 100%;
    }

    body {
      background-image: url("Images/image1.jpg"); /* */
      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      background-attachment: fixed;
      background-repeat: repeat-y;

      flex-direction: column;
      color: white;
      font-family: Arial, sans-serif;
    }

    .overlaytext {
      font-size: 4rem;
      margin-bottom: 20px;
      text-align: center;
      padding: 40px;
      text-align: center;
      color: rgb(10, 30, 79);
      width: 100%;
    }

    .overlaytext h1 {
      font-size: 4rem;
      margin-bottom: 20px;
    }

    .overlaytext p {
      font-size: 1.5rem;

```

```

        margin-bottom: 30px;
    }
</style>

<div class="weathersection">
    <section class="overlaytext">
        <h1>Your Weather Buddy</h1>
        <p>Weather analytics and activity recommendation</p>
    </section>

    <section>
        <form class="weatherForm">
            <input type="text" class="cityInput" placeholder="Enter
Location">
            <button type="submit">Get Weather</button>
        </form>
    </section>

    <div class="resultRow">
        <div class="card" style="display: none"></div>
        <!-- summaryBox id for js function -->
        <div class="QuickSummary" id="summaryBox" style="display:none">
            <h2 class="summaryTitle">Quick Summary</h2>
            <p id="avgTemp"></p>
            <p id="rainRisk"></p>
            <p id="activitySuggest"></p>

            <hr class="summaryDivider">
            <h3 class="summarySubTitle">Suggested Activities</h3>
            <ul id="recommendedList" class="recommendedList"></ul>
        </div>
    </div>

    <!--compare location-->

    <div class="QuickSummary">
        <h2 style="color: rgb(10, 30, 79);">Plan your trips</h2>
        <p>Compare two places and see which one is better for outdoor
activities.</p>

        <div class="compareForm">
            <input type="text" class="cityA" placeholder="City A (e.g., Kuala
Lumpur)">
            <input type="text" class="cityB" placeholder="City B (e.g.,
Langkawi)">
            <button type="button" class="compareBtn">Compare</button>
        </div>

        <!-- js function use compareResukt class to show output -->
        <div class="compareResult"></div>

        <script src="index.js"></script>
    </div>
</body>
</html>

```

Page2.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>OpenWeather Application</title>
  <link rel="stylesheet" href="../css/style.css">
  <link rel="stylesheet" href="../css/dashboard.css">
</head>

<body>

<header>
  <div class="container-header">
    <div class="logo">Weather<span>Cast</span></div>
    <nav>
      <a href="../index.html">Home</a>
      <a href="../Pages/page2.html" class="btn">Activity Recommendation
Manager</a>
    </nav>
  </div>
</header>

<h1>Activity Recommendation</h1>

<div id="activityForm">
  <input type="text" id="activityName" placeholder="Activity Name" required>

  <select id="activityType">
    <option value="Indoor">Indoor</option>
    <option value="Outdoor">Outdoor</option>
  </select>

  <!-- for add button -->
  <button id="addButton" type="button">Add</button>
</div>

<p id="feedbackMsg"></p>
<div id="activityList"></div>

<script src="../js/page2.js"></script>

</body>
</html>

```

dashboard.css

```

/* base default styling */
html, body{
  margin: 0;
  padding: 0;
  height: 100%;
}

body{
  font-family: Arial, sans-serif;
  background-color: hsl(0, 0%, 95%);
}

/* font */
h1{
  color: rgb(10, 30, 79);
  font-size: 3.5rem;
  margin-top: 0;
  margin-bottom: 25px;
}

p{
  color: rgb(10, 30, 79);
  font-size: 1.5rem;
  margin: 5px 0;
}

/* weatherbox */
.weathersection{
  width: 100%;
  padding: 50px 20px 40px;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: stretch;
}

.layout{
  display: flex;
  width: 100%;
  min-height: 100vh;
}

/* get weather form */
.weatherForm{
  margin: 20px;
  display: flex;
  justify-content: center;
  align-items: center;
}

.cityInput{ /* user input */
  padding: 10px;
  font-size: 2rem;
  font-weight: bold;
  border: 2px solid hsla(0, 0%, 20%, 0.3);
  border-radius: 10px;
  margin: 10px;
}

```

```

    width: 300px;
  }

  /* submit button */
  button[type="submit"]{
    padding: 10px 20px;
    font-weight: bold;
    font-size: 2rem;
    background-color: rgb(10, 30, 79);
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
  }

  button[type="submit"]:hover{
    background-color: rgb(10, 30, 79);
  }

  /* to make the box side by side */
  .resultRow{
    display: flex;
    flex-direction: row !important;
    gap: 30px;
    width: 100%;
    align-items: stretch;
    flex-wrap: nowrap;
  }

  /* weather box */
  .card{
    background: rgba(255,255,255,0.3);
    backdrop-filter: blur(10px);
    flex: 2;

    color: white;
    border: 1px solid rgb(179, 177, 177);
    padding: 40px;
    border-radius: 15px;
    box-shadow: 0 8px 20px rgba(0,0,0,0.2);

    width: 100%;
    max-width: 1100px;

    display: flex;
    flex-direction: column;
    align-items: left;
    gap: 15px;

    background-blend-mode: overlay;
    background-color: rgba(255,255,255,0.45);
  }

  /* flex overrides so card will not force wrap */
  .resultRow .card{
    flex: 2;
    width: auto;
    max-width: none;
    min-width: 0;
  }

```

```

}

/* font style in the card */
.cityDisplay, .tempDisplay{
  font-size: 3.5rem;
  font-weight: bold;
  color: rgb(10, 30, 79);
  margin-bottom: 25px;
}

.humidityDisplay{
  color: rgb(10, 30, 79);
  font-weight: bold;
  margin-bottom: 25px;
}

.descDisplay{
  font-style: italic;
  font-weight: bold;
  font-size: 2rem;
}

.errorDisplay{
  font-size: 2.5rem;
  font-weight: bold;
  color: rgb(10, 30, 79);
}

/*quick summary box*/
.QuickSummary{
  flex: 1.2;
  background: rgba(255,255,255,0.3);
  backdrop-filter: blur(10px);
  padding: 25px;
  border-radius: 15px;
  box-shadow: 0 8px 20px rgba(0,0,0,.2);
}

.summaryTitle{
  color: rgb(10,30,79);
  margin-bottom: 15px;
  font-size: 1.6rem;
  font-weight: 700;
}

/* added one line to divide the section */
.summaryDivider{
  margin: 15px 0;
  border: none;
  border-top: 1px solid rgba(10,30,79,0.2);
}

/* for suggested activities font text */
.summarySubTitle{
  color: rgb(10,30,79);
  font-size: 1.2rem;
  margin-bottom: 10px;
}

```

```

/* Suggested activities list in summary */
.recommendedList{
  list-style: none;
  padding-left: 0;
  margin: 0;
}

.recommendedList li{
  background: rgba(255,255,255,0.35);
  padding: 10px 12px;
  border-radius: 10px;
  margin-bottom: 10px;
  color: rgb(10,30,79);
  font-weight: 600;
}

/* page 2 styling */
#activityList{
  margin-top: 12px;
}

.activityItem{
  display: flex;
  align-items: center;
  justify-content: flex-end;
  gap: 12px;

  padding: 10px 12px;
  margin: 8px 0;

  border: 1px solid #ddd;
  border-radius: 8px;
  background: #fff;
}

.activityItem button{
  padding: 6px 10px;
  border: 1px solid #bbb;
  border-radius: 6px;
  background: #f7f7f7;
  cursor: pointer;
}

.activityItem button:first-of-type{
  margin-left: auto;
}

/* compare location styling */
.compareForm{
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin-top: 10px;
}

.compareForm input{
  flex: 1;
  min-width: 220px;
  padding: 10px;
}

```

```
    font-size: 1rem;
  }

  .compareForm button{
    padding: 10px 16px;
    cursor: pointer;
  }
```


style.css

```

body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
}
input, button {
    margin: 5px;
}

html, body {
    margin: 0;
    padding: 0;
    height: 100%;
}

.window {
    padding: 0 !important;
}

/* reset and default design */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
    line-height: 1.6;
    color: #333;
    background: #f8f9fa;
}

/* Container & Button */
.container {
    width: 100%;
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 15px;
}

.btn {
    display: inline-block;
    padding: 10px 30px;
    background: rgb(10,30,79);
    color: #fff;
    border-radius: 4px;
    font-weight: 600;
    transition: .3s;
    text-decoration: none;
}

/* header */
header {
    background: #fff;
    box-shadow: 0 2px 10px rgba(0,0,0,.1);
    position: sticky;

```

```
    top: 0;
    z-index: 1000;
    width: 100%;
}

header .container-header{
    max-width: none; /* to make it full size */
}

.container-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 15px 0;
}

.logo {
    font-size: 28px;
    font-weight: 700;
    color: #ff6b6b;
    padding-left: 3%;
}

.logo span {
    color: #4ecdc4;
}

nav a {
    margin-left: 20px;
    color: #333;
    font-weight: 600;
    text-decoration: none;
}
```

Assessment Rubric

ATTRIBUTES	CRITERIA	POOR (1 mark)	FAIR (2 marks)	GOOD (3 marks)	VERY GOOD (4 marks)	EXCELLENT (5 marks)	Mark Obtained
	1. Create desktop application using electron framework with the integration	The application is an extensive collection and rehash of other people's ideas, products, and images. There is no evidence of new thought.	The application is somewhat a collection and rehash of other people's ideas, products, and images. There is little evidence of new thought or inventiveness.	The application is a minimal collection or rehash of other people's ideas, products, and images. There is a few evidence of new thought or inventiveness.	The application shows a lot of evidence of originality and inventiveness.	- The application shows significant evidence of originality and inventiveness . - Most of the content and many of the ideas are fresh, original, and inventive.	

Reproduce and Process Information	of the given API. Your application needs to output the information required above.	• Unable to display all the required data from the API and does not fulfill the requirements.	• Able to display all the required data from the API but no manipulation of data.	• Able to display extra data from the API beyond the application requirements and have basic manipulation	• Able to manipulate and display extra data from the API beyond the application requirements with the description in the report		
		• The data from	• The data	• The data from	• Able to utilize the data	• Able to utilize the data	

		the API does not reflect the whole purpose of the application developed.	from the API is sufficient but does not reflect the whole purpose of the application developed.	the API is meaningful but does not reflect the purpose of the application developed.	fetched from the API to come up with meaningful functionalities that reflect the purpose of the application developed. Come up with extra idea for the application developed that enhances the user experience or adds value to the application.	
--	--	--	---	--	---	--

	2. Implement CRUD (create, read, update, delete) process to the application to make your application more meaningful.	• Able to create only 2 of the CRUD processes. • No feedback for the CRUD process. • The design for data input is poor	• Able to create only 3 of the CRUD processes. • No feedback for the CRUD process. • The design for data input is good with some room for	• Able to perform all the CRUD processes. • No feedback for the CRUD process. • Well-designed data input for CRUD process.	• Able to perform all the CRUD processes. • No feedback for the CRUD process. • Well-designed data input for CRUD process.	• Able to perform all the CRUD processes. • Appropriate feedback for the CRUD process. • Well-designed and user-friendly data input for CRUD process.	
--	---	--	---	--	--	---	--

			improvement				
--	--	--	-------------	--	--	--	--

	3. Apply HTML and CSS for user interface and provide evidence for application.	• Text - All text used is too small to view or the font type is wrongly chosen. • Graphics - Graphics seem randomly chosen, are of low quality, OR distract the reader.	• Text – Some of the text used is too small to view or the font type is wrongly chosen. • Graphics - Graphics seem randomly chosen, are of low quality, OR distract the reader.	• Text - Most text used is clear but does not describe the content well. • Graphics - Graphics are related to the theme/purpose of the application and are of excellent quality.	• Text - All text used is clear but does not describe the content well. • Graphics - Graphics are related to the theme/purpose of the application, are of excellent quality and enhance reader interest or understanding	• Text - All text used is clear and able to describe the content well. • Graphics - Graphics are related to the theme/purpose of the application, are thoughtfully cropped, are of high quality and enhance reader interest or understanding.	
--	--	--	--	---	---	--	--

Curate		• Not able to curate for required content. • Layout - The HTML	• Limited curation for required content. • Layout - The HTML	• Satisfactory curation for required content. • Layout -The HTML	• Good curation for required content. • Layout - The HTML	• Excellent curation for required content. • Layout - The HTML	
	4. GUI Elements:	elements in the application are cluttered looking	elements in the application is messy, may appear busy or boring.	elements are suitable.	elements are suitable and usable.	elements are well structured, attractive, and usable layout.	
	. Apply GUI elements that assist users in using application.	or confusing.		• Navigation Links allow the reader to move from page to page, but some links seem to be missing.	• Navigation Links are labelled and allow the user to easily move from page to page.	• Navigation Links are clearly labelled, consistently placed, and allow the user to easily move from page to page.	
		• Navigation Links do not take the reader to the sites/ pages described. User typically feels lost.	• Navigation Links seem to be missing and don't allow the user to easily navigate.				

	ii. The application's 'look and feel' is attractive and informative.	- The application needs polish in its visual design and is not appropriate for the target audience. Color	- The application needs polish in its visual design, but it is still appropriate for the target audience. Color	The application mostly follows good visual design principles (e.g.: alignment, contrast, easily read	The application demonstrates good visual design principles (e.g.: alignment, contrast, easily read	The application clearly demonstrates good visual design principles (e.g.: alignment, contrast,	
--	--	---	---	--	--	--	--

		Choice of colors and combinations are not suitable.	Choice of colors and combinations do not match the concept of the application.	text) and is appropriate for the target audience. • Color Choice of colors and combination s match the concept of the application.	text) and is appropriate for the target audience. • Color Appropriate colors used to produce an atmosphere that expresses the concept of the application.	easily read text) and is appropriate for the target audience. • Color Appropriate colors used to produce an atmosphere that expresses the concept of the application.	
Convey	. Produce a report on your application. Include the following: . Overview of the application and its purpose. Include explanation of the information	• The overview of the application and is vague	• The overview of the application and is very brief.	• The overview of the application and its purposes is clearly described.	• Include explanation of the information retrieved from the API and CRUD process with its purposes.		

	retrieved from the given API and CRUD process.						
	ii. Description of the functionalities and features of the developed application with print screen of	The description of the functionalities and	The description of the functionalities and	The description of the functionalities and features	The description of the functionalities and features	The description of the functionalities and features is complete with	

	the pages and explanation.	features is poor and unorganized. • Incomplete print screen of the application.	features is very brief. • Incomplete print screen of the application.	is complete. • Complete screenshot of the pages.	is complete with extensive information. • Complete screenshot of the pages.	extensive information. • Complete screenshot of the pages and the labelling of the features in the application.	
	iii. Program codes of your system	• HTML, CSS, and JavaScript codes attached are not complete. • The codes are hardly read.	• HTML, CSS, and JavaScript codes attached are complete. • The codes are hardly read.	• HTML, CSS, and JavaScript codes attached are complete. • The codes are readable but not organized.	• HTML, CSS, and JavaScript codes attached are complete. • The codes are readable and organize.	• HTML, CSS, and JavaScript codes attached are complete and include comments for the important parts of the codes. • The codes are	

						readable and organized.	
	6. Submit files in GitHub.	Does not submit complete electron files in GitHub	Completely submit all the electron file in GitHub.				
Total Marks Earned							/50
Total Percentage (40%)							/40%